

PaseQR — Boletos digitales con QR en la nube

Proyecto Integrador · Desarrollo en la Nube · Otoño 2026 · Prof. Marcela Rosales, ITESO · Fase 1: Planteamiento del proyecto

Equipo: David Valdez · Vittorio Catino · Juan Pablo Gutiérrez | **Repositorio:** <https://github.com/davidvaldez/paseqr-boletos-nube>

1. Problema y aplicación

Problema. Los eventos pequeños y medianos (conciertos universitarios, conferencias, torneos, fiestas) suelen venderse con hojas de cálculo, transferencias y listas impresas. Eso genera **sobreventa**, **boletos falsificados o reutilizados** (una captura de pantalla se comparte fácilmente), **filas lentas** en la entrada y ningún dato confiable de asistencia. Las plataformas comerciales resuelven esto, pero cobran comisión por boleto y no permiten control sobre los datos.

Aplicación. PaseQR es una aplicación web donde el **organizador** crea un evento con cupo; el **asistente** compra o se registra y recibe un **boleto PDF con un código QR firmado**; el **staff** valida la entrada desde su celular escaneando el QR; y el organizador recibe **recordatorios automáticos** y un **reporte de asistencia** al terminar el evento. El pago se simula (no se integra una pasarela real).

¿Por qué en la nube?

- **Demanda en picos:** casi no hay tráfico la mayor parte del tiempo, pero hay ráfagas cuando abre la preventa y en la hora de entrada. Lambda y API Gateway escalan solos y se paga por invocación; un servidor propio tendría que dimensionarse para el pico y estaría ocioso el resto del tiempo.
- **Consistencia con concurrencia:** las escrituras condicionales de DynamoDB garantizan que nunca se vendan más boletos que el cupo, aunque cientos de personas compren al mismo tiempo.
- **Desacoplamiento y tolerancia a fallas:** SQS absorbe los picos de generación de PDFs, reintenta automáticamente y manda a una DLQ lo que falla, sin que el comprador tenga que esperar.
- **Acceso desde cualquier lugar:** la puerta del evento solo necesita un celular con internet; los boletos quedan guardados de forma durable en S3.
- **Tareas programadas y notificaciones sin servidores encendidos** (EventBridge + SNS), con un costo que cabe en la capa gratuita de AWS para el volumen del proyecto.

2. Flujos end-to-end

Flujo	Entrada y validación	Lógica de negocio	Resultado observable
1. Compra y emisión de boleto Propio de la nube	El asistente elige evento y cantidad e ingresa nombre y correo. Se validan formato, que el evento exista y esté a la venta, y el límite de boletos por persona.	Una escritura condicional en DynamoDB aparta el lugar solo si hay cupo (sin sobreventa). Se crea el boleto <i>PENDIENTE</i> y se encola en SQS . Un Lambda consumidor genera el PDF con QR firmado (HMAC), lo guarda en S3 y marca el boleto como <i>EMITIDO</i> .	Boleto PDF descargable en “Mis boletos” (URL prefirmada de S3) y un correo de confirmación por SNS . Si la emisión falla, el mensaje llega a la DLQ.
2. Check-in con QR en la puerta	El staff entra con el PIN del evento y escanea el QR con la cámara del celular. Se valida la firma del QR con la llave guardada en Secrets Manager.	Se verifica que el boleto exista, pertenezca a ese evento y no se haya usado. Una actualización condicional lo marca como <i>USADO</i> con fecha y hora, lo que evita dobles entradas aunque dos personas lo escaneen a la vez.	Pantalla verde o roja con el motivo (válido, ya usado, inválido u otro evento) y el contador de asistencia en vivo del evento actualizado.

3. Recordatorios y reporte de asistencia	El organizador define la fecha del evento y su correo (con validación de fechas futuras). Se dispara cada hora con EventBridge Scheduler.	Un Lambda busca eventos que empiezan en 24 h y publica el recordatorio. Para los eventos que ya terminaron calcula boletos vendidos contra asistentes, la tasa de asistencia y la hora pico de entrada.	Correo de recordatorio a los asistentes y un reporte CSV en S3 que se envía al organizador por SNS (enlace de descarga).
---	---	---	---

3. Servicios de AWS

Servicio	Uso en PaseQR	¿Por qué este servicio?
AWS Lambda	Lógica de los 3 flujos, del consumidor de la cola y de la tarea programada.	Escala automáticamente con los picos y cuesta cero cuando no hay tráfico. No hay servidores que administrar.
Amazon API Gateway	API HTTP (/eventos, /compras, /checkin) que usa el frontend.	Punto de entrada administrado con CORS, <i>throttling</i> para proteger el backend en la preventa y métricas de latencia y errores.
Amazon DynamoDB	Tablas de Eventos y Boletos, con un índice por correo y otro por evento.	Las escrituras condicionales atómicas evitan la sobreventa y el doble check-in. Respuesta en milisegundos y modo <i>on-demand</i> para tráfico irregular.
Amazon S3	Hosting del frontend estático, boletos PDF y reportes CSV.	Almacenamiento durable y barato. Las URLs prefirmadas permiten descargar un boleto sin hacerlo público.
Amazon SQS	Cola de emisión de boletos con DLQ.	Desacopla la compra de la generación del PDF: absorbe ráfagas y reintenta sin perder mensajes.
Amazon SNS	Confirmación de compra, recordatorios, reporte al organizador y avisos de alarmas.	Mensajería <i>pub/sub</i> administrada con entrega por correo y filtros por suscriptor.
Amazon CloudWatch	Logs, métricas personalizadas, alarmas y dashboard.	Da visibilidad del sistema completo en un solo lugar y está integrado de forma nativa con todos los servicios anteriores.

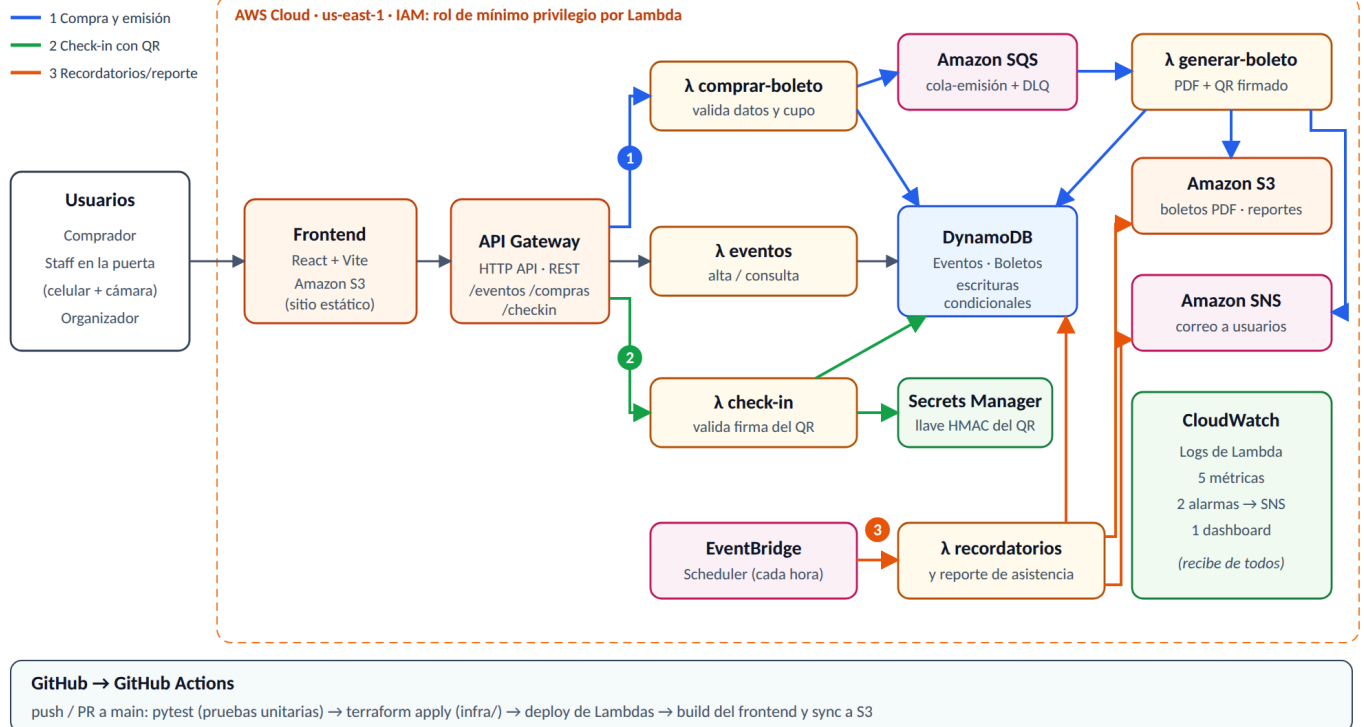
Complementarios: **IAM** (un rol de mínimo privilegio por Lambda), **Secrets Manager** (llave para firmar los QR, así un QR no se puede falsificar aunque se conozca el formato) y **EventBridge Scheduler** (disparo programado del flujo 3).

4. Roles del equipo

Integrante	Rol	Responsabilidades principales
David Valdez	Flujo 1 + Frontend	Compra, cola SQS y generación del PDF con QR. Aplicación React (catálogo, compra, "Mis boletos", escáner y panel del organizador) desplegada en S3.
Vittorio Catino	Flujo 2 + CI/CD	Check-in, firma y validación de QR con Secrets Manager. Workflows de GitHub Actions: pruebas, terraform apply y despliegue automático al integrar a main .
Juan Pablo Gutiérrez	Flujo 3 + Monitoreo	Scheduler, recordatorios y reporte CSV. Métricas personalizadas, alarmas y dashboard de CloudWatch.

La infraestructura se escribe en **Terraform** (*infra*/) entre los tres, con un módulo por flujo. Cada integrante escribe las pruebas unitarias (pytest) de su flujo y todos trabajan con *Pull Requests* revisados por otro integrante.

5. Boceto del diagrama de arquitectura



Frontend estático en S3 → API Gateway → Lambdas. El flujo 1 es asíncrono vía SQS, el flujo 2 valida contra DynamoDB y Secrets Manager, y el flujo 3 es programado por EventBridge. Todo reporta a CloudWatch y se despliega con GitHub Actions.

6. Visión del entregable final

Al final del semestre PaseQR estará **100% desplegada en AWS** y se podrá recrear desde el repositorio con Terraform. En el demo se creará un evento, se comprarán boletos (incluida una ráfaga de compras simultáneas para probar que no hay sobreventa), se escanearán los QR desde un celular (válido, repetido y falsificado) y se mostrará el reporte de asistencia. Cada `merge` a `main` ejecutará pruebas y desplegará automáticamente.

- **5 métricas:** boletos emitidos, check-ins válidos y rechazados (personalizadas), errores de Lambda, latencia p95 de API Gateway y antigüedad del mensaje más viejo en SQS.
- **2 alarmas (aviso por SNS al equipo):** tasa de errores 5xx de la API mayor a 5% en 5 minutos, y cualquier mensaje en la DLQ (un boleto que no se emitió).
- **1 dashboard** con el estado del sistema y la actividad de ventas y entradas en tiempo real.
- **Plan:** Fase 2 (15 oct): flujos 1 y 2 y frontend en la nube con Terraform (≥80%). Revisión del 12 nov: flujo 3, pruebas y CI/CD completos. Fase 3: monitoreo, pruebas de carga y pulido.